Forwarding and Routing

IT5100D AY25/26 S2

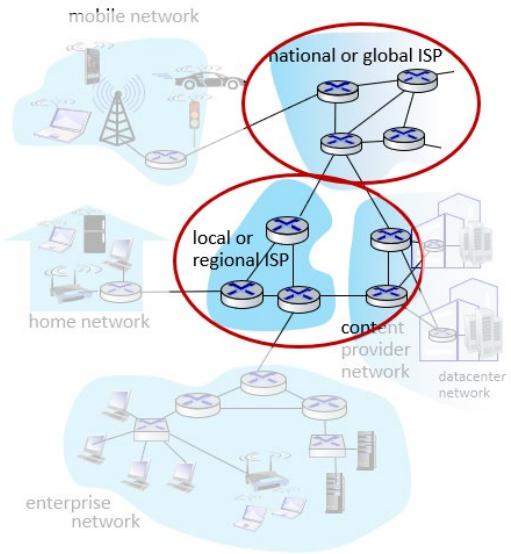
NUS | Computing
National University of Singapore

© Copyright National University of Singapore. All Rights Reserved.

Packet Switching

- The network core: a mesh of interconnected routers
- Data are transmitted over the network in chunks, called packets
- **Packet-switching:** packets are routed independently throughout the network, from source to destination

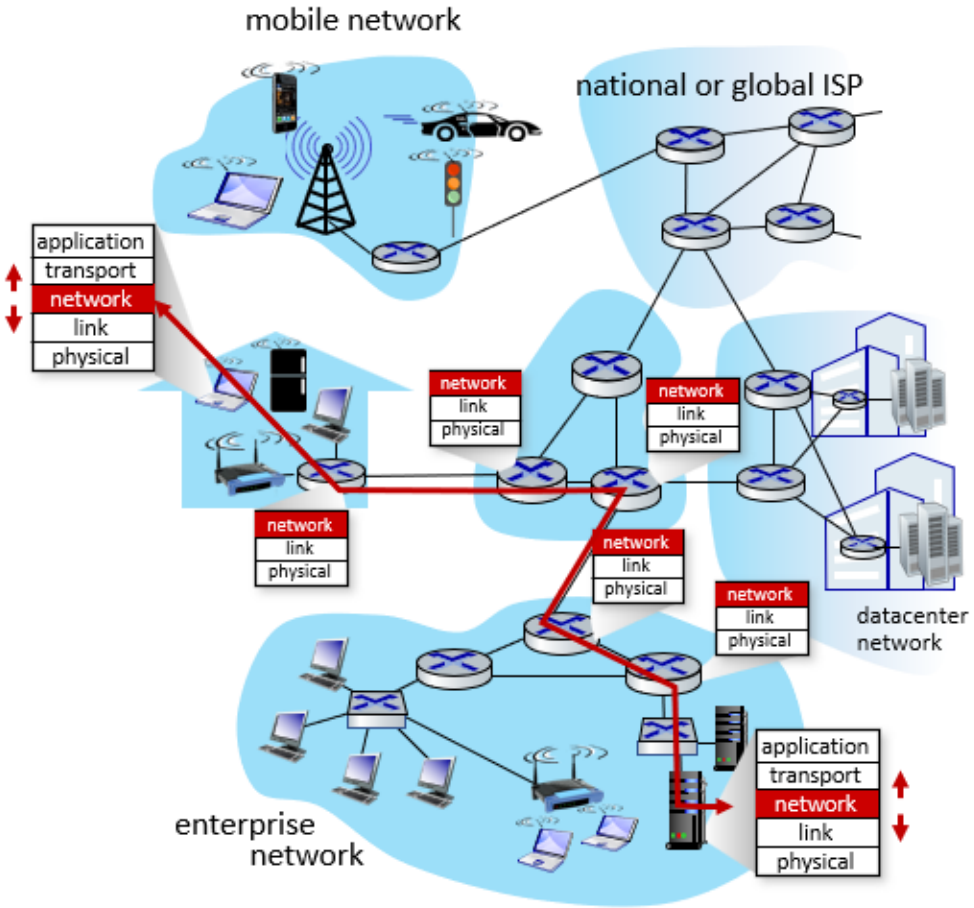
- Most computer networks are packet-switched
- As opposed to *circuit-switched*: dedicated, pre-routed, line between the two parties (e.g., traditional telephone networks)
- Packet switching is considered more efficient overall, as network links are used on-demand rather than pre-allocated



Network Layer Services

Router actions:

- Examines header fields in all datagrams passing through it
- Moves datagrams from input ports to output ports, to transfer datagrams along the end-to-end path



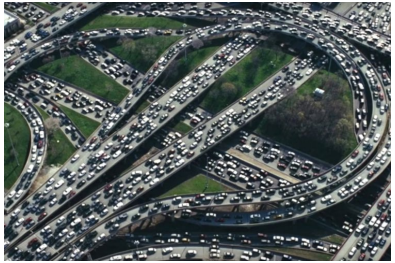
Network Layer – Two Key Functions

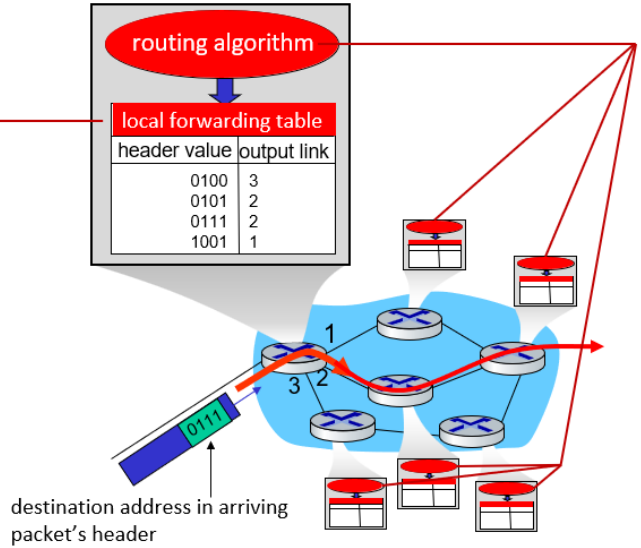
Data plane

Forwarding:

- *Local* action: move arriving packets from router's input link to appropriate router output link

Analogy: getting through an interchange





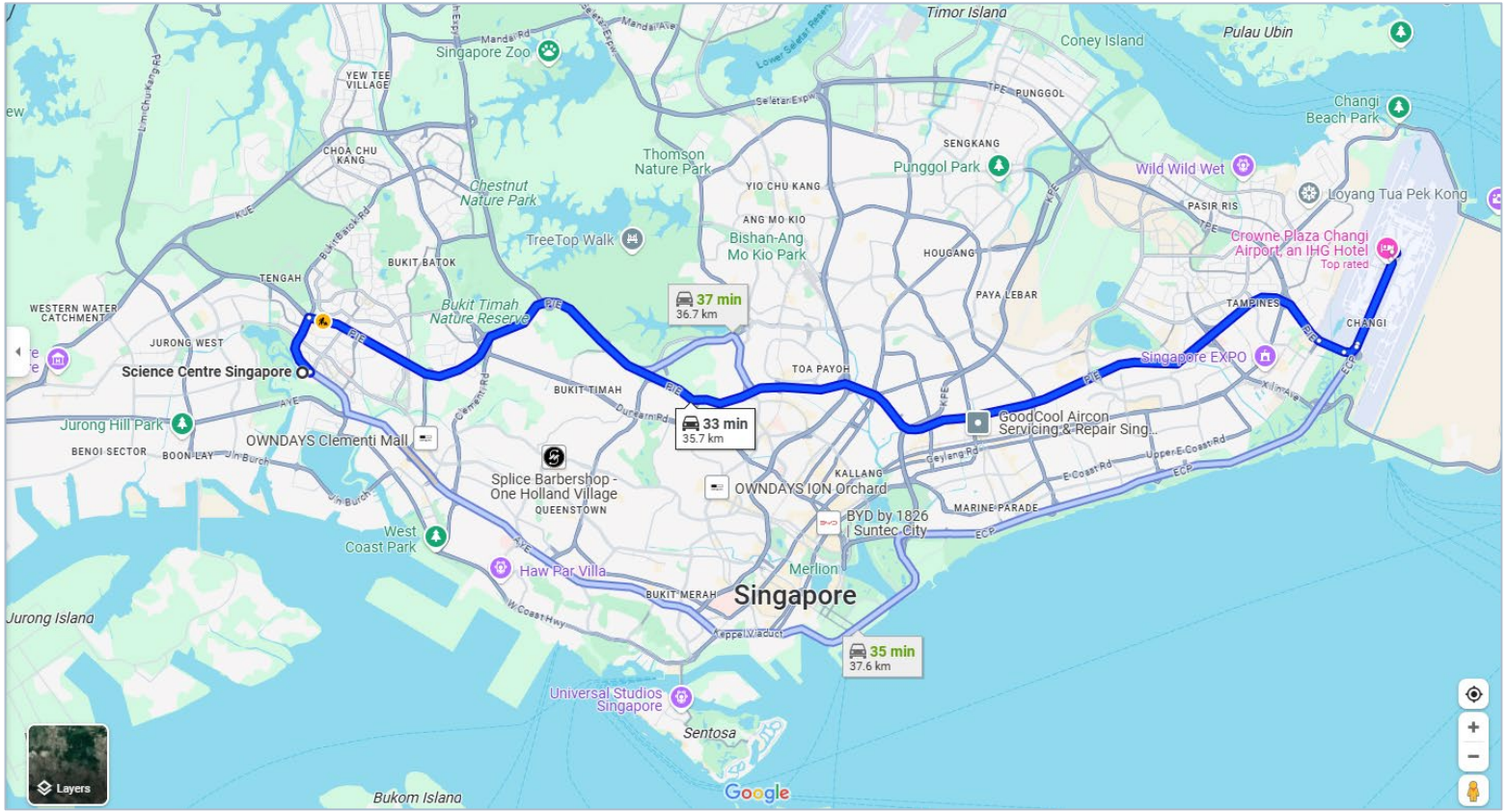
Control plane

Routing:

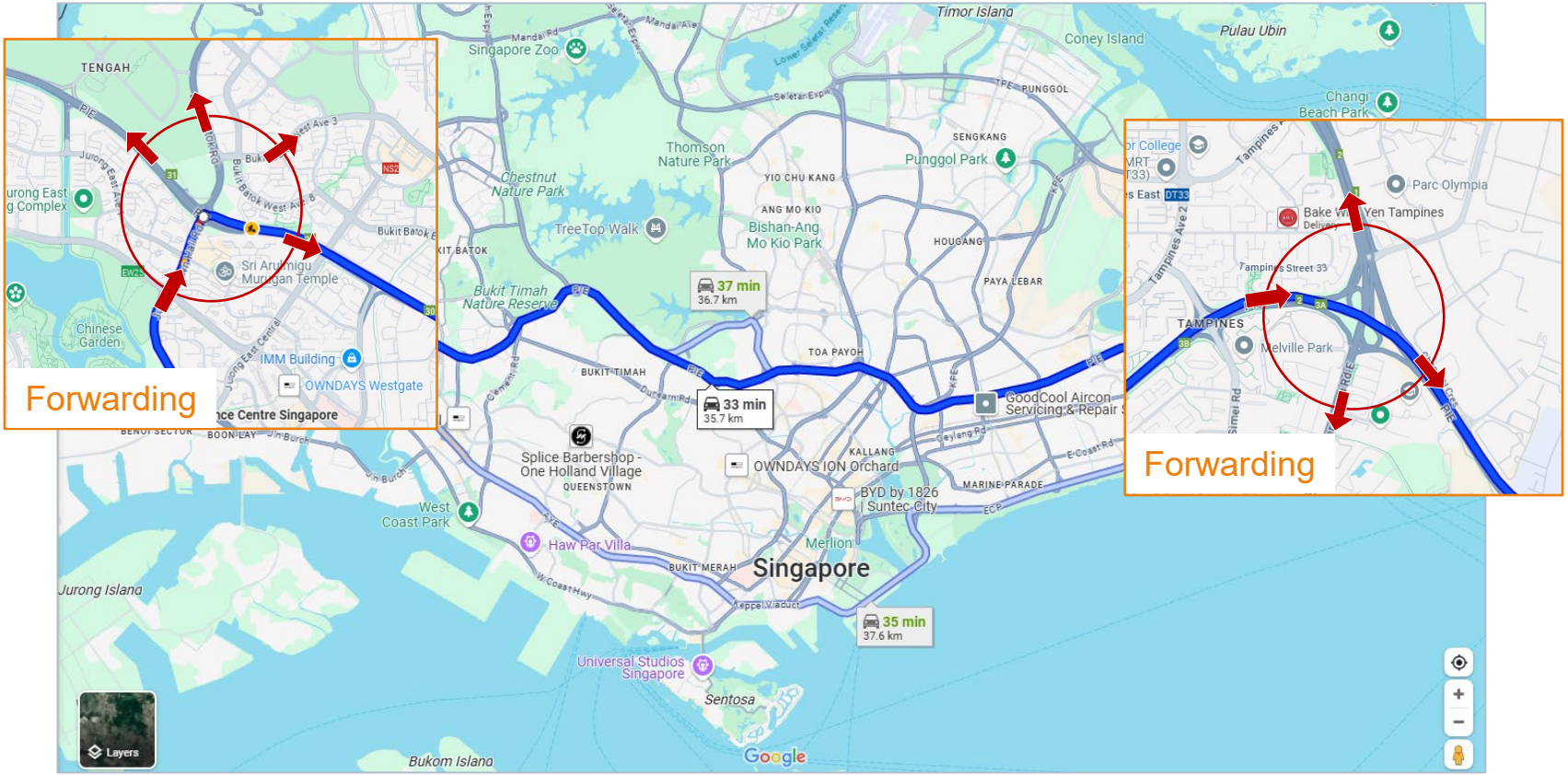
- *Global* action: determine source-destination paths taken by packets

Analogy: planning trip from source to destination

Routing Analogy

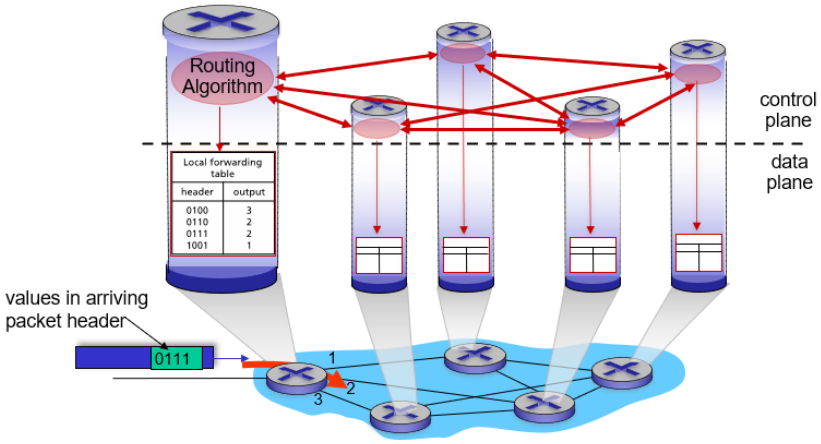


Forwarding Analogy

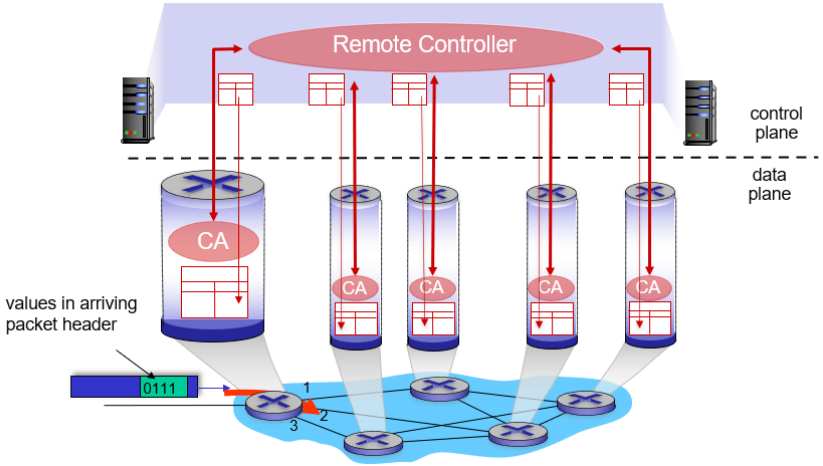


Data/Control Plane Approaches

- Traditional routing algorithms
  - Implemented in routers, together with data-plane forwarding
  - Algorithm components in every router interact with each other to compute forwarding table



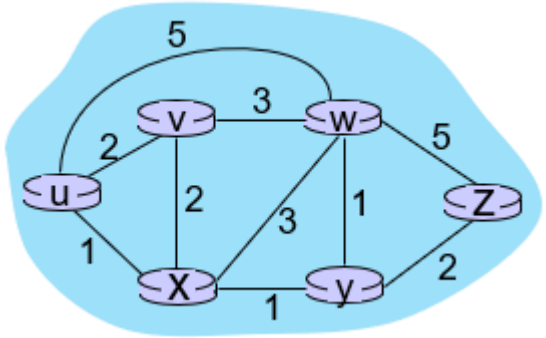
- Software-defined networking (SDN)
  - Separate control plane from data plane
  - Logically centralized control
  - Remote controller computes and installs forwarding table in routers



Routing Protocols

- Goal: determine “good” paths (or routes) from sending hosts to receiving host, through network of routers
  - Path: sequence of routers packets traverse from given initial source host to final destination host
  - “Good”: least “cost”, “fastest”, “least congested”
- In practice: policy issues may introduce more rules
- Routing: a “top-10” networking challenge!

Graph abstraction with link costs



Routing Algorithm Classification

Global

- All routers have *complete* topology, link cost info
- Each node broadcast to all nodes: link-state packets containing identities + costs of its attached links

→ *"Link state"* algorithms

Decentralized

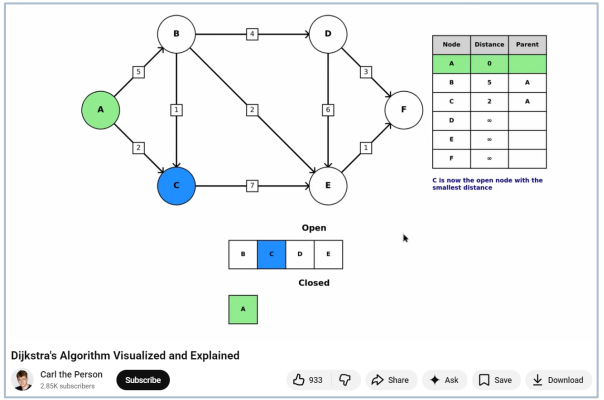
- Iterative process of computation, exchange of info with neighbors
- Routers initially only know link costs to attached neighbors

→ *"Distance vector"* algorithms

Link State Routing Algorithm

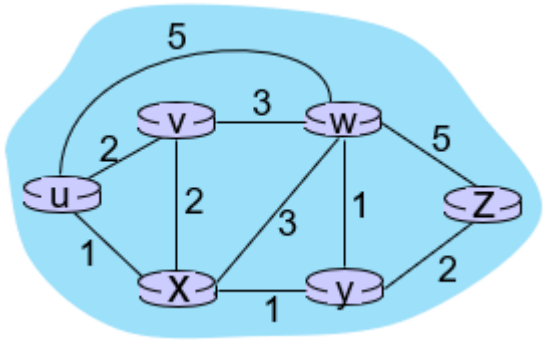
Dijkstra's link-state routing algorithm

- Centralized: network topology, link costs known to *all* nodes
  - Accomplished via "link state broadcast"
  - All nodes have same info
- Computes least cost paths from one node ("source") to all other nodes
  - Gives *forwarding table* for that node
- Iterative: after *k* iterations, know least cost path to *k* destinations



Djikstra Algorithm Visualized and Explained

Dijkstra's Algorithm



N: set of routers = {u, v, w, x, y, z}

c(a, b): cost of direct link connecting a and b

D(v): *current* cost estimate of least-cost path from source to destination v

Algorithm to find least-cost paths from a node u to all other nodes in N

```
Initialization:
  N' = {u}
  for all nodes v
  if v is a neighbor of u
  then D(v) = c(u, v)
  else D(v) = ∞

Loop:
  find w not in N' such that D(w) is a minimum
  add w to N'
  update D(v) for each neighbor v of w and not in N':
    D(v) = min( D(v), D(w)+ c(w, v) )
until N'= N
```

Distance Vector Algorithm

- Based on *Bellman-Ford (B-F) equation* (dynamic programming)
  - Similar intuition to Dijkstra's algorithm: take the minimum of paths over all neighbouring nodes

Bellman-Ford equation

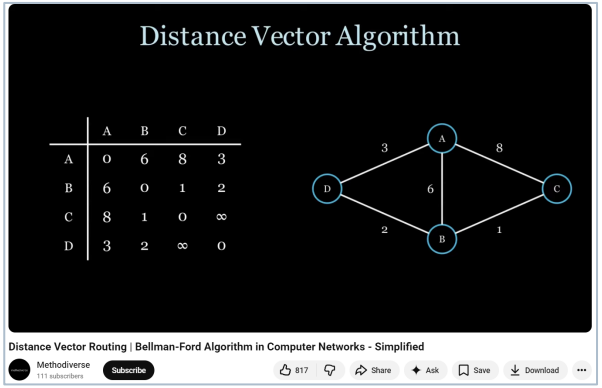
Let $D_x(y)$: cost of least-cost path from x to y.
Then:

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

v's estimated least-cost-path cost to y

min taken over all neighbors v of x

direct cost of link from x to v



[Distance Vector Routing - Simplified]

Distance Vector Algorithm

Key idea:

- From time-to-time, each node sends its own *distance vector estimate* to neighbours
- When *x* receives new DV estimate from any neighbour, it updates its own DV using B-F equation
- Under minor, natural conditions, the estimate $D_x(y)$ *converge to the actual least cost* $d_x(y)$

Each node:

Wait for (change in local link cost or msg from neighbour)

Recompute DV estimates using DV received from neighbour

If DV to any destination has changed, *notify* neighbours

Distance Vector Algorithm

Iterative, asynchronous

- Each local iteration caused by:
  - Local link cost change
  - DV update message from neighbour

Distributed, self-stopping

- Each node notifies neighbours only when its DV changes
  - Neighbours then notify their neighbours – only if necessary
  - No notification received, no actions taken!